PROCEDIMIENTO DE IMPLEMENTACIÓN

FaceLit

Despliegue, migraciones, verificación y operación

CORTE	PLATAFORMA VIGENTE
27 de octubre de 2026	VPS Hostinger · Docker Compose · Caddy · PostgreSQL 16 + pgvector

Equipo: Valery Sinaí Trujillo Quintero · María José Rodríguez Oyola · Mariana Valenzuela Penagos
Programa: SENA · Análisis y Desarrollo de Software
Instructor: José de Jesús Motta

Esta edición reemplaza el despliegue descrito anteriormente con Render/Vercel por el estado reportado en Hostinger. Describe evidencia técnica y pendientes manuales por separado; no incluye secretos ni datos personales.

1. Arquitectura implementada

La capa pública usa Caddy para servir la web y hacer proxy de las rutas API y WebSocket. Backend Spring, servicio facial FastAPI y PostgreSQL permanecen en la red privada de Docker Compose. Solo la capa web publica los puertos 80 y 443.

Componente	Tecnología / función	Exposición
Web y proxy	Caddy sirve el frontend y enruta API y WebSocket.	HTTPS público; puertos 80/443.
Backend	Java 17 · Spring Boot 3.4.1. Autenticación, permisos, reglas académicas, sesiones y asistencia.	Interno; se accede mediante Caddy.
Servicio facial	Python 3.11 · FastAPI. Liveness, embeddings y verificación facial.	Interno; lo coordina el backend por HTTP.
Base de datos	PostgreSQL 16 + pgvector; utilizada por backend y servicio facial.	Red interna; no se publica al cliente.

Repositorios y referencias al corte

Repositorio	Rama / HEAD local
FaceLit (cliente Expo)	dev · 5f7d6fe
FaceLit-Backend	dev · 2ab0bcd
micro-facial	dev · 1e12c6c
FaceLit-DB	main · 41a063f

Los repositorios separan responsabilidades, pero el backend Java continúa siendo una aplicación monolítica modular. El servicio facial es independiente; ambos servicios de aplicación comparten PostgreSQL.

2. Secuencia de implementación

Orden	Acción documentada	Control de salida
1	Preparar paquete de despliegue sin secretos y revisar versiones/configuración.	Confirmar los SHA, variables privadas y artefactos exactos.
2	Crear/verificar respaldo antes de modificar el esquema; validar los changesets Liquibase.	Conservar evidencia del respaldo y validación antes de update.
3	Aplicar migraciones a PostgreSQL en orden coordinado antes de iniciar backend.	45 changesets reportados; aplicación sin pendientes.
4	Construir/actualizar imágenes Docker del VPS y activar servicios.	Backend inicia con Hibernate validate; salud de DB y facial verificada.
5	Validar HTTPS, rutas API y WebSocket a través de Caddy.	Web/login 200, health facial 200, admin sin auth 401, WebSocket 101.
6	Ejecutar aceptación manual con cuentas/datos autorizados y comprobar persistencia.	Pendiente para login real, CSV, SMTP, cámara y asistencia física.

Migración H-50 de nomenclatura

Se corrigió la diferencia entre consumidores Java y columnas heredadas. La expansión añadió `role_name`, `description` y `assigned_at`, copió valores y mantuvo compatibilidad dual mediante triggers. La fase de contrato, después de actualizar consumidores, retiró `name_rol`, `descripcion` y `asigned_at`, normalizó categorías y corrigió una FK. La migración llegó a FaceLit-DB/main mediante PR #63; existen rollbacks por etapa. No aplicar contracción mientras queden consumidores antiguos.

La secuencia reportada pasó de 39 a 45 changesets. Respaldar y ejecutar en el orden del changelog; no editar changesets históricos ya aplicados.

3. Estado de la base de producción

La aplicación usa PostgreSQL 16 con pgvector. Las migraciones se administran con Liquibase desde FaceLit-DB; Hibernate valida el esquema al iniciar el backend. La actualización de octubre incluyó respaldo, despliegue de migraciones y limpieza transaccional autorizada de datos operativos.

Dato posterior a la actualización	Estado reportado
Cuenta de coordinación	Se conservó una cuenta de coordinación, sus credenciales, roles/permisos, configuración y aceptación legal.
Datos operativos	Se eliminaron instructores, aprendices, programas, fichas, referencias faciales y eventos previos.
Importación de datos locales	No se importó la base de datos local.
Datos para demostración	Preparar un conjunto autorizado antes de la sustentación y comprobar relaciones después de importar.
Backups	El respaldo contiene información sensible y permanece en el VPS; no se distribuye con este documento.

La operación de producción no debe confundirse con el conjunto de desarrollo. Antes de la demostración, crear solo los datos autorizados necesarios y evitar incluir documentos personales, fotografías o credenciales en capturas públicas.

Restauración de respaldo

La creación de un respaldo se reporta; su restauración en un entorno separado sigue pendiente. No modificar producción para ensayar recuperación. Registrar versión de PostgreSQL, cambios de esquema, verificación de integridad y resultado de consultas posteriores.

4. Verificaciones realizadas y su alcance

Comprobación	Resultado reportado	Límite
Inicio de servicios	PostgreSQL y servicio facial saludables; backend inicia con Hibernate validate.	Confirma arranque técnico; no aceptación del usuario.
Web/HTTPS	Ruta web/login responde 200.	No demuestra login funcional con cuenta real.
API protegida	Endpoint administrativo sin autenticación responde 401.	Comprueba rechazo sin token, no matriz completa de autorización.
WebSocket	Handshake responde 101 a través del proxy.	No certifica todos los mensajes/eventos de negocio.
Microservicio	123 pruebas aprobadas; una prueba opcional de PostgreSQL omitida.	Retos y señales sintéticas; no validación facial poblacional.
Frontend	120 pruebas en 13 suites; seis scripts de regresión.	No certifica cámara física, SMTP ni todo el lint global.
Integración HTTP	19 comprobaciones de retos y recorridos de registro, autorización, asistencia, matriz y CSV sobre DB desechable.	Inferencia controlada y correo simulado; no producción/cámara física.

Los conteos pertenecen a ejecuciones concretas de HU-45 y no se deben sumar a informes de otras ramas/fases. Las pruebas de integración comprueban contratos y coordinación; no miden rendimiento bajo carga, precisión biométrica ni entrega SMTP real.

5. Cliente móvil y flujo facial

El cliente Expo puede exportarse para web y compilarse para Android con EAS, pero el despliegue web actual está publicado en el VPS Hostinger. La aceptación del APK en el dispositivo de sustentación no está documentada como completa.

El flujo facial actual envía la captura al backend Spring por HTTPS. Spring valida la solicitud y coordina FastAPI por HTTP interno; el servicio facial evalúa la secuencia y consulta referencias en PostgreSQL. Spring aplica reglas de sesión y asistencia y devuelve el resultado.

Configuración facial vigente	Valor
Retos	MOVE_UP, MOVE_DOWN, MOVE_CLOSER, MOVE_AWAY.
Secuencia	Dos retos distintos; siete imágenes por reto.
Umbral observado	0.65 de similitud; no equivale a 65 % de precisión.
Límite de aceptación	Sin prueba reportada con cámara real/personas diversas ni estudio FAR/FRR.

Revisar la configuración EAS antes de compilar/publicar: la configuración local reportada contiene un dominio/API diferente del dominio web Hostinger actualmente comprobado. No declarar el APK listo hasta confirmar URL, build SHA, instalación y recorrido en el dispositivo objetivo.

Integración heredada

El README del frontend conserva referencias EXPO_PUBLIC para URL/clave facial. El servicio de captura revisado utiliza la API del backend. Retirar la documentación/configuración obsoleta y revisar el bundle para confirmar que no se publiquen credenciales.

6. Seguridad y operación

Control operativo	Estado y criterio
Secretos	Credenciales faciales/DB/JWT/SMTP permanecen en configuración privada del servidor; no registrarlas en Git, PDF, logs o capturas.
Red	HTTPS desde navegador; backend, servicio facial y PostgreSQL por red interna. Solo Caddy publica 80/443.
Migraciones	Respaldo antes de cambios, Liquibase coordinado y backend con validación JPA. Confirmar rollbacks antes de cambio de esquema.
Roles y API	Validar permisos por endpoint y alcance de ficha/sesión; ocultar menús no reemplaza autorización del servidor.
SMTP	Configuración técnica presente, pero entrega real y notificaciones no se han aceptado manualmente.
Disponibilidad	Health checks reportados; sin medición formal de disponibilidad, latencia o capacidad.

Los documentos anteriores de Render/Vercel se conservan solo como antecedente histórico. Su configuración, costos, suspensión de planes y dominios no describen el despliegue vigente y no se deben copiar al próximo release.

7. Procedimiento para una siguiente actualización

1) Registrar los SHA exactos de frontend/backend/facial/DB y revisar cambios de configuración. 2) Proteger credenciales y crear respaldo verificable. 3) Validar y aplicar migraciones del repositorio DB antes de iniciar backend dependiente. 4) Construir/actualizar imágenes en Hostinger y activar servicios internos. 5) Comprobar salud, HTTPS, API protegida y WebSocket. 6) Ejecutar login, CSV, SMTP, cámara/asistencia y consulta persistente con datos autorizados. 7) Restaurar respaldo en entorno separado. 8) Si se distribuye APK, confirmar el dominio/API, instalar el artefacto exacto y documentar resultados. 9) Adjuntar logs/artefactos redactados sin datos sensibles.

Conclusión: FaceLit está desplegado técnicamente en Hostinger con web/proxy público y servicios internos. Migraciones, checks HTTP y suites automatizadas proporcionan evidencia técnica específica. La liberación operativa completa sigue condicionada a aceptación manual, restauración ensayada, revisión del APK/configuración y pruebas de correo/cámara reales.

Fuentes de actualización: Implementacion_Realizada_2026-10-27.md; Pruebas_Evidencias_Actualizadas.md; DESPLIEGUE_HOSTINGER_2026-10-27.md; repositorios en los SHA de portada.